< DAKER · AI MENTORING SYSTEM >

DAKER AI 멘토링
— 개념·프로세스·UI 한눈에 보기 —

동의 게이트 → CP 세션 → SSE 채팅 → 리포트 · 피드백

4 Checkpoint + 자유 채팅 · Gemini · SSE 스트리밍 · Drizzle/PostgreSQL

해커톤 모드 + 팀 플래그 + Office Hours 3단 제어

ONE-LINE DEFINITION

해커톤 전 기간 동안, 필요한 순간에 정해진 질문 흐름으로 AI 멘토가 붙어 완주를 돕는 시스템

적용 위치

- 참가자: **팀 워룸 내 AI 멘토링 탭**
- 관리자: **해커톤 관리자 콘솔**
- 컴포넌트: `client/src/components/mentoring/`

세션 타입

- CP1 팀 셋업 / CP2 아이디어 리뷰
- CP3 중간 점검 / CP4 최종 리뷰
- 자유 채팅 (힌트 기반, 무제한)

기술 스택

- **AIProvider** 인터페이스 추상화
- **Gemini** (learning-chat 공용)
- **SSE** 스트리밍 + Drizzle ORM

5	4	24/7	p95<10s
세션 타입	신규 테이블	응답 가능 (Office Hours 제한 가능)	AI 응답 SLO

PROBLEM / SOLUTION

기존 해커톤 멘토링의 4가지 한계를 동시에 해결

문제

- 멘토 역량 편차 — 같은 CP를 받아도 팀별 품질 격차
- 사각지대 — 스코프·리스크가 본선 직전까지 노출 안 됨
- 멘토 부족 — 본선 집중 시간대 기술 자문 과부하
- B2B 상품화 공백 — DAKER 플랫폼 부가가치 창출 여지

해결

- **품질 균등화** — 5종 시스템 프롬프트로 고정 품질
- **구조화 발견** — 강제 질문 흐름이 스코프/리스크 노출
- **24/7 접근** — AI 멘토 상시 가용, SSE 실시간 응답
- **B2B 상품화** — 멘토링 모드 설정 · 비용 한도 관리

핵심 인사이트

"사람 멘토의 보조"가 아니라 **"구조화된 AI 세션 + 사람 멘토 하이브리드"**를 전제로 설계.
AI가 먼저 스코프·리스크를 노출시키고, 사람 멘토는 고차원 판단에 집중.

04

시스템 아키텍처

공통 AIProvider + 멘토링 라우터 + 5 세션 타입 + SSE 스트리밍으로 구성된 단일 모노리스 구조

LAYERED STRUCTURE

4층 구조 — 공통 AI · 서비스 · 라우터 · UI

```
// 공통 AI 계층
server/services/ai-provider.ts
 ├─ AIProvider (interface)
 │   ├─ chat(messages, config)
 │   ├─ chatStream(messages, config)   // SSE
 │   └─ countTokens(text)
 └─ GeminiProvider

// 서비스 계층
server/services/
 ├─ mentoring-prompt.ts   // 5종 프롬프트
 ├─ mentoring-context.ts  // 컨텍스트 빌더
 └─ mentoring-office-hours.ts
```

```
// 라우터 계층
server/routes/mentoring/
 ├─ sessions.ts     // CRUD + 잠금
 ├─ messages.ts     // SSE 스트리밍
 ├─ reports.ts      // 리포트 조회
 └─ admin.ts        // 설정·통계

// 스키마
shared/schema/mentoring.ts   // 4 테이블
shared/schema/hackathons.ts  // mode/config/enabled

// UI
client/src/components/mentoring/*
client/src/hooks/useMentoring*
```

설계 원칙

기존 패턴 재사용: learning-chat.ts의 Gemini 초기화·Rate Limiter·Provider 패턴을 그대로 공용화해서 멘토링과 공유.
최소 침습: 기존 파일 변경 최소, 신규 파일 추가 위주. **Provider 추상화**: Gemini/OpenAI 교체 가능 구조.

CHECKPOINT & FREE CHAT

5종 시스템 프롬프트 — CP별 AI 역할이 명확히 다르다

타입	라벨	AI 역할	톤	핵심 지시
`team_setup`	**팀 셋업**	Team Coach	격려 + 전략적	역할 분배·일정 최적화. **프로필 확인형** 첫 질문으로 팀원 정보 기반 추천
`idea_review`	**아이디어 리뷰**	Idea Mentor	도전적 + 건설적	**리프레이밍**. "실제로 만들고 있는 건 X" 패턴으로 스코프 노출
`mid_check`	**중간 점검**	Tech Mentor	실용적 + 긴급	남은 기간 대비 스코프 컷을 **대담하게 제안**. 기능 컷 제안 필수
`final_review`	**최종 리뷰**	Submit Reviewer	꼼꼼 + 임팩트	평가 루브릭 기준 **체크리스트**. quick-win 2~3개 집중 가이드
`free_chat`	**자유 채팅**	General Mentor	친근 + 힌트	**정답 직접 제공 금지**. 코드 50줄 이하. 언제든 호출 가능

QuestionItem[]
강제 질문 흐름

soloAlt
솔로 참가자 대체 질문

zeroCodeAlt
코드 0줄 대체 질문

followUpAction
AI 후속 행동 지침

CONSENT GATE

동의 게이트는 클라이언트 · 서버 양쪽에서 강제

STEP 1
탭 진입
useMentoringConsentStatus()
→ consented 여부 조회

→

STEP 2
ConsentDialog
미동의 시 모달 오픈.
섹션별 체크박스 → 저장

→

STEP 3
CP / Free Chat 시작
CheckpointCard 클릭
OR [자유 채팅] 버튼

→

STEP 4
POST /sessions
세션 생성 · 팀 컨텍스트 빌드
→ MentoringChat 뷰로 이동

에러 분기

- **403 CONSENT_REQUIRED** → 서버가 재차 동의 요구 → ConsentDialog 재오픈
- **409 Conflict** → 동일 CP에 진행 중 세션 존재 → `data.sessionId`로 **기존 세션 이어가기**
- **429** → Rate Limit · 일일 토큰 초과 → 사용자 안내

pendingAction 패턴

동의 모달이 열리기 전 사용자가 시도한 행동(CP 시작 · 자유 채팅)을 `pendingAction` state에 저장.
동의 완료 후 자동으로 해당 행동을 재실행 — **흐름 끊김 없이 이어가기**.

CONSENT CONTENT

동의서 8개 필수 섹션 + 1 선택 · v1.0 (2026-04-14)

#	key	제목	필수	핵심 내용
1	`collect`	개인정보 수집·이용	필수	팀장·팀원 성명·역할·기술스택·가용시간·대화·리포트. 해커톤 종료 후 1년 보존
2	`overseas`	국외 이전	필수	Google LLC(미국)로 메시지 전송. 모델 학습 미사용 (Vertex AI/Gemini API 정책)
3	`teamShare`	팀원 정보 공유	필수	팀장이 팀원 정보 AI 제공 시 팀원 전원 공유. 팀원 고지 완료 확인
4	`forbidden`	금지 입력	필수	주민번호·건강·금융·제3자 정보·유해 콘텐츠 입력 금지. 위반 책임 본인
5	`disclaimer`	AI 한계·면책	필수	참고용 · 법률/의료/재무 자문 아님 · 환각 가능 · DAKER/주최측 면책
6	`ip`	저작권·산출물	필수	입력 메시지 저작권 본인 귀속. AI 리포트 자유 활용, 제출물에 자동 포함 X
7	`adminAccess`	운영자 열람	필수	운영자는 리포트만 접근. 대화 전문 열람은 별도 고지
8	`minorConsented`	연령 확인	필수	만 14세 이상 또는 법정대리인 동의
9	`marketing`	마케팅 활용	선택	체크 안 해도 멘토링 이용 가능. 사례 연구 · 홍보 활용 동의 (선택)

🏷️ 버전 관리

- `CURRENT_MENTORING_CONSENT_VERSION = 'v1.0'`
- 단일 소스: `shared/schema/mentoring.ts`
- 서버·클라이언트 동일 import → drift 방지
- 버전↑ 시 기존 사용자 재동의 모달 (법령 변경 안내)

🗄️ 저장 스키마 — mentoring_consents

`id · userId(FK) · version · consentedAt · revokedAt`
`ipAddress · userAgent · sectionsJson(JSONB)`

유니크: `(userId, version)` — 버전당 1회 기록

🔐 API · 게이트

- `GET /api/mentoring/consents/me`
- `POST /api/mentoring/consents`
- 세션 생성 시 서버 검증 → **403 CONSENT_REQUIRED**
- 테스트: **9/9 통과** (consents 7 + gate 2)

SERVER-SENT EVENTS

메시지 전송 → SSE chunk 수신 → 리포트 자동 생성

```
// 1. 사용자 입력 (≤2000자)
POST /api/mentoring/sessions/:id/messages
  body: { content: "..." }

// 2. 서버 처리
check dailyTokenLimit → 429 if exceed
save user message (role='user')
build context:
    systemPrompt + previousReports
    + recentTurns(10)
call AIProvider.chatStream(ctx, config)

// 3. SSE 스트림
data: {"chunk":"안녕하세요..."}
data: {"chunk":" 팀의 목표는..."}
data: {"done":true,"tokenCount":421}
```

```
// 4. 메시지 저장
save assistant message (tokenCount)
update session.tokenUsage += N

// 5. 리포트 자동 생성
if (session.type != 'free_chat'
   && allQuestionsAnswered) {
  generateReport(session, messages)
  → insert into mentoring_reports
  → share to War Room
}

// 미지원 클라이언트용 JSON 폴백
POST /.../messages?stream=false
→ { content, tokenCount, done }
```

2,000	50,000	10/min	5/min	30s
입력 자수 제한	팀당 일일 토큰	메시지 Rate Limit	세션 Rate Limit	API 타임아웃

COMPONENT MOCK

CheckpointCard — Completed · Available · Locked 3종

✅ COMPLETED

CP1. 팀 셋업

2026-04-13 14:30

역할 분배 · 일정 초안 수립

★★★★☆ 4.0

리포트 보기

🔵 AVAILABLE

CP2. 아이디어 리뷰

이전 CP 완료

아이디어 리프레이밍 · 차별점 검토

멘토링 시작

🔒 LOCKED

CP3. 중간 점검

이전 단계 완료 필요

(CP2 · 아이디어 리뷰)

상태 판정 로직 — `getCheckpointStatus()`

① 완료 세션 있음 → **completed**

② 진행 중 세션 있음 → **in_progress**

③ 첫 번째 CP (cpIndex === 0) → **available**

④ 이전 CP 완료 세션 있음 → **available**

⑤ 이외 → **locked**

COMPONENT MOCK

MentoringChat — SSE 수신 실시간 렌더

CP1. 팀 셋업 질문 3 / 5

안녕하세요 팀 "알파"! 프로필을 보니 홍길동님이 백엔드, 이영희님이 디자인을 맡으셨네요. 두 분이 지금 가장 걱정되는 리스크 한 가지씩만 알려주실 수 있을까요?

백엔드는 인증 연동이 걱정이고, 디자인은 반응형 레이아웃이 어려울 것 같습니다.

좋은 자기 진단입니다. 그럼 이 두 리스크를 해커톤 3일 안에 체크하려면, 각각 어떤 작업을 24시간 내로 끝내야 한다고 보시나요?

답변을 입력하세요... (최대 2000자) 전송

구성 요소

- **ChatHeader** — CP명 + 진행률 (질문 N/M)
- **ChatMessages** — user/assistant 구분 렌더
- **ChatInput** — 2000자 제한, Enter 전송
- **ChatActions** — [세션 포기] / [리포트 보기]

훅

- `useMentoringSession` — React Query 기반 세션 상태
- `useMentoringSSE` — 스트림 chunk 수신 훅
- `useMentoringConsent` — 동의 상태 & 저장

REPORT & FEEDBACK

세션 완료 → 리포트 자동 생성 → 피드백 수집

MentoringReport

- **ReportHeader** — CP명 · 점수(0~100) · 생성일
- **ReportContent** — `contentJson` 구조화 렌더
- **ReportRecommendations** — `recommendations[]`
- War Room에 **자동 공유** — 팀 전원 열람 가능

Score: 78 / 100

✓ 역할 분배 구체적

⚠ 인증 PoC 24h 내 완료 필요

✓ 마일스톤 3개 정의됨

MentoringFeedback

- **StarRating** — ★ 1~5점
- **CommentInput** — 선택 입력
- 유니크 제약: `(sessionId, userId)`
- 세션당 1회만 제출 가능

★★★★☆

"질문이 구조적이어서 도움됐어요.
조금 더 구체적 예시가 있으면 좋겠습니다."

리포트 데이터 모델 (mentoring_reports)

`id` · `sessionId(FK)` · `reportType` · **`contentJson (JSONB)`** · `summaryText` · **`score(0–100)`** · **`recommendations (JSONB)`** · `createdAt`

MULTI-LEVEL SWITCHES

관리자 제어 — 해커톤 · 팀 · 시간대 3단 스위치

① 해커톤 단위

`hackathons.mentoring_mode`
`hackathons.mentoring_config` (JSONB)

- 모드: disabled optional required
- 활성 CP 목록 선택
- 자유 채팅 on/off
- LLM 모델 · 일일 토큰 한도

② 팀 단위 (2026-04-15)

`hackathons.mentoring_enabled` flag
(feat: 8aec9ca 참가팀별 플래그)

- 참가팀별 on/off 가능
- 해커톤 모드가 enabled여도 팀 플래그 false면 차단
- 민감 팀 · 파일럿 팀만 선별 가능

③ Office Hours

`services/mentoring-office-hours.ts`

- 시간대 제한 운영 (예: 10:00~22:00)
- 시간 외 호출 시 안내 메시지
- 비용 ·멘토 운영 시간 통제

관리자 API — server/routes/mentoring/admin.ts

```
PATCH /api/admin/hackathons/:id/mentoring  ·  GET /api/admin/hackathons/:id/mentoring/stats
GET /api/admin/hackathons/:id/mentoring/teams  →  팀별 진행 현황 · 토큰 사용량 · 만족도
```

SCHEMA & LIMITS

4개 테이블 + 비용 · Rate · 데이터 보존 가드레일

테이블	핵심 필드	제약
`mentoring_sessions`	checkpointType · status · currentQuestionIndex · reportJson · tokenUsage	팀당 동일 CP `in_progress` 1개
`mentoring_messages`	role · content · modelUsed · questionIndex · tokenCount · metadataJson	session FK cascade
`mentoring_reports`	reportType · contentJson · summaryText · score(0–100) · recommendations	session FK cascade
`mentoring_feedback`	rating(1~5) · comment	`(sessionId, userId)` unique

💰 비용 제어

- 팀당 일일 토큰 50,000
- 메시지 10 req/min/user
- 세션 5 req/min/team

⚠️ 장애 대응

- 타임아웃 30초 → 재시도 1회
- 연속 3회 실패 → 관리자 알림
- 서킷 브레이커 + 일시 중지

🗄️ 데이터 보존

- 대화: 해커톤 종료 90일 후 익명화
- 리포트: 1년 보존 후 통계만 유지
- 개인정보 분리 저장

DACON · DAKER

구조화 · 품질균등 · 24/7 —
AI 멘토가 팀의 완주를 보증합니다

공통 AIProvider · 5 세션 타입 · SSE 스트리밍 · 3단 관리자 제어

docs/01-plan/features/ai-mentoring.plan.md · docs/02-design/features/ai-mentoring.design.md